Name: KRIHSNA KOUSHIK

THT NO: 2303A51589

Batch: 19

Task 1: Hospital Management Database Prompt:

Generate a complete SQL solu2on for a Hospital Management System by crea2ng a normalized database schema with three tables: Doctors, Pa2ents, and Appointments. Include primary keys, foreign keys, and proper constraints such as NOT NULL and UNIQUE, with appropriate data types compa2ble with SQLite. Then write SQL queries using joins to list all appointments for a specific doctor using doctor_id, retrieve full pa2ent history using pa2ent_id, and count the total dis2nct pa2ents treated by each doctor. Also include sample INSERT statements for tes2ng and a few SELECT queries to verify the data. The output should be clean, well-structured, and executable in SQLite without errors.

Code:

```sql
-- SQLite
-- DROP old tables (important if you ran before)
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Doctors;
DROP TABLE IF EXISTS Patients;

-- Create Doctors table
CREATE TABLE Doctors ( doctor_id INTEGER PRIMARY KEY AUTOINCREMENT, first_name
    TEXT NOT NULL, last_name TEXT NOT NULL, specialty TEXT NOT NULL,
    UNIQUE (first_name, last_name) );

-- Create Patients table
CREATE TABLE Patients ( patient_id INTEGER PRIMARY KEY AUTOINCREMENT, first_name TEXT
    NOT NULL, last_name TEXT NOT NULL, date_of_birth TEXT NOT NULL,
    UNIQUE (first_name, last_name, date_of_birth) );

-- Create Appointments table
CREATE TABLE Appointments ( appointment_id INTEGER PRIMARY KEY AUTOINCREMENT, doctor_id
    INTEGER NOT NULL, patient_id INTEGER NOT NULL, appointment_date TEXT NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id) );
-- Insert data
INSERT INTO Doctors (first_name, last_name, specialty) VALUES
```

```
('John', 'Doe', 'Cardiology'),
('Jane', 'Smith', 'Neurology');

INSERT INTO Patients (first_name, last_name, date_of_birth) VALUES
('Alice', 'Johnson', '1980-05-15'),
('Bob', 'Brown', '1990-08-22');

INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES
(1, 1, '2024-07-01'),
(1, 2, '2024-07-02'),
(2, 1, '2024-07-03');

        OUTPUT (IMPORTANT)
-- TEST
SELECT * FROM Doctors;
SELECT * FROM Patients;
SELECT * FROM Appointments;

--
SELEC   ired Queries
T a.     a.appointment_id, d.first_name, d.last_name, p.first_name, p.last_name,
        ntment_date
FROM    pointments a
JOIN    octors d ON a.doctor_id = d.doctor_id
JOIN    tients p ON a.patient_id = p.patient_id
WHERE   a.doctor_id = 1;
SELECT
FROM    a.appointment_id,     d.first_name,     d.last_name,     a.appointment_date
JOIN    pointments a
WHERE   ctors d ON a.doctor_id = d.doctor_id
        a.patient_id = 1;
SELECT
FROM     d.doctor_id, d.first_name, d.last_name, COUNT(DISTINCT a.patient_id) Doctors
LEFT
GROUP   IN Appointments a ON d.doctor_id = a.doctor_id
BY         d.doctor_id;
CREATE TABLE test123 (id INTEGER);  name
SELECT ROM sqlite_master;
```

Output:

SQL 1 / 1 1 - 2 of 2

appointment_id	first_name	last_name	first_name	last_name	appointment_date
1	John	Doe	Alice	Johnson	2024-07-01
2	John	Doe	Bob	Brown	2024-07-02

SQL 1 / 1 1 - 2 of 2

appointment_id	first_name	last_name	appointment_date
1	John	Doe	2024-07-01
3	Jane	Smith	2024-07-03

SQL 1 / 1 1 - 2 of 2

doctor_id	first_name	last_name	COUNT(DISTINCT a.patient_id)
1	John	Doe	2
2	Jane	Smith	1

ExplanaIon:

The database is designed with three related tables where the Appointments table links Doctors and Pa2ents using foreign keys, ensuring proper normaliza2on. SQL joins are used to combine data across tables to retrieve appointments, pa2ent history, and aggregated counts efficiently.

Task 2: **Real-Time ApplicaIon: Online Shopping Database Prompt:**

Generate a complete SQL solu2on for an E-commerce (Online Shopping) database system by crea2ng a normalized schema with the tables Users, Products, Orders, and OrderDetails. Include primary keys, foreign keys, appropriate data types, and constraints such as NOT NULL and UNIQUE, ensuring compa2bility with SQLite. Establish proper rela2onships between tables and maintain at least Third Normal Form (3NF). Then write SQL queries using joins to retrieve all orders placed by a specific user using user_id, find the most purchased product based on total quan2ty sold, and calculate total revenue for a given month. Include sample INSERT statements for tes2ng and SELECT queries to verify data. Also suggest brief normaliza2on improvements and query op2miza2on techniques. The output should be clean, well-structured, and fully executable without errors.

Code:

```sql
-- SQLite
-- SQLite E-commerce Database (FINAL CLEAN)

PRAGMA foreign_keys = ON;

-- RESET (important to avoid duplicate/exists
errors)
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products; DROP TABLE
IF EXISTS Users;

-- Create Users table
CREATE TABLE Users ( user_id INTEGER PRIMARY
    KEY AUTOINCREMENT, username TEXT NOT NULL
    UNIQUE, email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP );

-- Create Products table
CREATE TABLE Products ( product_id INTEGER
    PRIMARY KEY AUTOINCREMENT, product_name TEXT
    NOT NULL, price REAL NOT NULL, stock INTEGER
    NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP );

-- Create Orders table
CREATE TABLE Orders ( order_id INTEGER PRIMARY
    KEY AUTOINCREMENT, user_id INTEGER NOT
    NULL,
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
```

```sql
);

-- Create OrderDetails table
CREATE TABLE OrderDetails ( order_detail_id INTEGER
    PRIMARY KEY AUTOINCREMENT, order_id INTEGER NOT
    NULL, product_id INTEGER NOT NULL, quantity
    INTEGER NOT NULL, price REAL NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- Insert data
INSERT INTO Users (username, email, password) VALUES
('john_doe', 'john@example.com', 'password123');

INSERT INTO Products (product_name, price, stock) VALUES
('Laptop', 999.99, 10),
('Mouse', 29.99, 50);

INSERT INTO Orders (user_id) VALUES (1);

INSERT INTO OrderDetails (order_id, product_id, quantity, price)
VALUES
(1, 1, 1, 999.99),
(1, 2, 2, 29.99);

-- TEST OUTPUT (you MUST see data)
SELECT * FROM Users;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM OrderDetails;

-- 1. Retrieve all orders by a user
SELECT o.order_id, o.order_date, od.product_id, od.quantity, od.price
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id WHERE
o.user_id = 1;

-- 2. Most purchased product
SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold FROM
Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id
ORDER BY total_quantity_sold DESC LIMIT
1;

-- 3. Total revenue in a given month
SELECT SUM(od.quantity * od.price) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
```

```sql
WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

Output:

SQL ▾ < 1 / 1 > 1 - 2 of 2

```
SELECT o.order_id, o.order_date, od.product_id, od.quantity, od.price
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE o.user_id = 1;
```

order_id	order_date	product_id	quantity	price
1	2026-03-18 13:18:04	1	1	999.99
1	2026-03-18 13:18:04	2	2	29.99

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id
ORDER BY total_quantity_sold DESC
LIMIT 1;
```

product_name	total_quantity_sold
Mouse	2

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
SELECT SUM(od.quantity * od.price) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

total_revenue
NULL

ExplanaIon:

The database uses mul2ple related tables where OrderDetails acts as a bridge between Orders and Products, ensuring proper normaliza2on and avoiding redundancy. SQL joins and aggregate func2ons are used to analyze user orders, iden2fy top-selling products, and calculate monthly revenue efficiently. **Task 3: Library Database Prompt:**

Generate a complete SQL solu2on for a Library Management System by crea2ng a normalized database schema with three tables: Books, Members, and Loans. Include primary keys, foreign keys, appropriate data types, and constraints such as NOT NULL and UNIQUE, ensuring compa2bility with SQLite. Establish proper rela2onships between tables and maintain at least Third Normal Form (3NF). Then write SQL queries using joins to retrieve all books currently issued (not yet returned), find overdue books where the loan date is more than 30 days from the current date, and count the number of books loaned by each member. Also suggest an indexing strategy to improve query performance. Include sample INSERT statements for tes2ng and ensure the output is clean, well-structured, and fully executable without errors.

Code:

```
-- SQLite
--Generate a complete SQL solution for a Library Management System by creating a normalized
database schema with three tables: Books, Members, and Loans. Include primary keys,
foreign keys, appropriate data types, and constraints such as NOT NULL and UNIQUE, ensuring
compatibility with SQLite. Establish proper relationships between tables and maintain at
least Third Normal Form (3NF). Then write SQL queries using joins to retrieve all books
currently issued (not yet returned), find overdue books where the loan date is more than
30 days from the current date, and count the number of books loaned by each
```

```
member. Also suggest an indexing strategy to improve query performance. Include sample
INSERT statements for testing and ensure the output is clean, well-structured, and fully
executable without errors.
-- Create the Books table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER NOT NULL,
    UNIQUE(Title, Author)
);
-- Create the Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Email TEXT NOT NULL UNIQUE
);
-- Create the Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER NOT NULL,
    MemberID INTEGER NOT NULL,
    LoanDate DATE NOT NULL,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Sample INSERT statements for testing
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('The Great Gatsby', 'F. Scott
Fitzgerald', 1925);
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('To Kill a Mockingbird', 'Harper
Lee', 1960);
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('1984', 'George Orwell', 1949);
INSERT INTO Members (FirstName, LastName, Email) VALUES ('John', 'Doe',
'john.doe@example.com');
INSERT INTO Members (FirstName, LastName, Email) VALUES ('Jane', 'Smith',
'jane.smith@example.com');
INSERT INTO Loans (BookID, MemberID, LoanDate) VALUES (1, 1, '2024-05-01');
INSERT INTO Loans (BookID, MemberID, LoanDate) VALUES (2, 2, '2024-05-15');
-- Query to retrieve all books currently issued (not yet returned)  SELECT
b.Title, b.Author, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- Query to find overdue books where the loan date is more than 30 days from the current
date
SELECT b.Title, b.Author, m.FirstName, m.LastName, l.LoanDate
```

```sql
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND julianday('now') - julianday(l.LoanDate) > 30;
-- Query to count the number of books loaned by each member
```

```sql
SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
-- Indexing strategy to improve query performance
CREATE INDEX idx_loans_bookid ON Loans(BookID);
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
CREATE INDEX idx_loans_loan_date ON Loans(LoanDate);
CREATE INDEX idx_books_title_author ON Books(Title, Author);
CREATE INDEX idx_members_email ON Members Email
```

Output:

SQLite

SQL 1 / 1 1 - 2 of 2

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

SQL 1 / 1 1 - 2 of 2

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

SQL 1 / 1 1 - 2 of 2

FirstName	LastName	BooksLoaned
John	Doe	1
Jane	Smith	1

ExplanaIon:

The system uses three related tables where Loans acts as a bridge between Books and Members, ensuring proper normaliza2on and avoiding redundancy. SQL joins and condi2ons are used to track issued books, detect overdue loans, and calculate borrowing ac2vity efficiently.

Task 4: OpImize Queries Prompt:

Analyze and op2mize SQL queries for be[er performance using a given example. Start with the original query that uses a subquery to retrieve books wri[en by UK authors, then rewrite it using an op2mized approach with joins. Ensure both queries return the same result set. Explain briefly how the op2mized query improves efficiency in terms of execu2on 2me and scalability. Suggest addi2onal op2miza2ons such as indexing relevant columns (e.g., author_id, country) and reducing unnecessary subqueries. Provide both the original and op2mized queries, along with a short valida2on step to confirm correctness of results. Ensure the output is clean, concise, and executable in SQLite.

Code:

```sql
-- SQLite
-- Analyze and optimize SQL queries for better performance using a given example

PRAGMA foreign_keys = ON;

-- Create Authors table
CREATE TABLE Authors ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, name TEXT NOT NULL, country
    TEXT NOT NULL
);

-- Create Books table
CREATE TABLE Books ( book_id INTEGER PRIMARY KEY
    AUTOINCREMENT, title TEXT NOT NULL, author_id INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(id)
);

-- Insert sample data
INSERT INTO Authors (name, country) VALUES
('Author A', 'UK'),
('Author B', 'USA'), ('Author C', 'UK');

INSERT INTO Books (title, author_id) VALUES
('Book 1', 1), ('Book
2', 2),
('Book 3', 3);

-- Original Query using subquery
SELECT title
FROM Books
WHERE author_id IN (
    SELECT    id    FROM
    Authors
    WHERE country = 'UK'
);

-- Optimized Query using JOIN
SELECT b.title
FROM Books b
JOIN Authors a ON b.author_id = a.id
WHERE a.country = 'UK';

-- Validation query (should return empty if both are same)
SELECT title
FROM Books
WHERE author_id IN (
    SELECT id FROM Authors WHERE country = 'UK' )
```

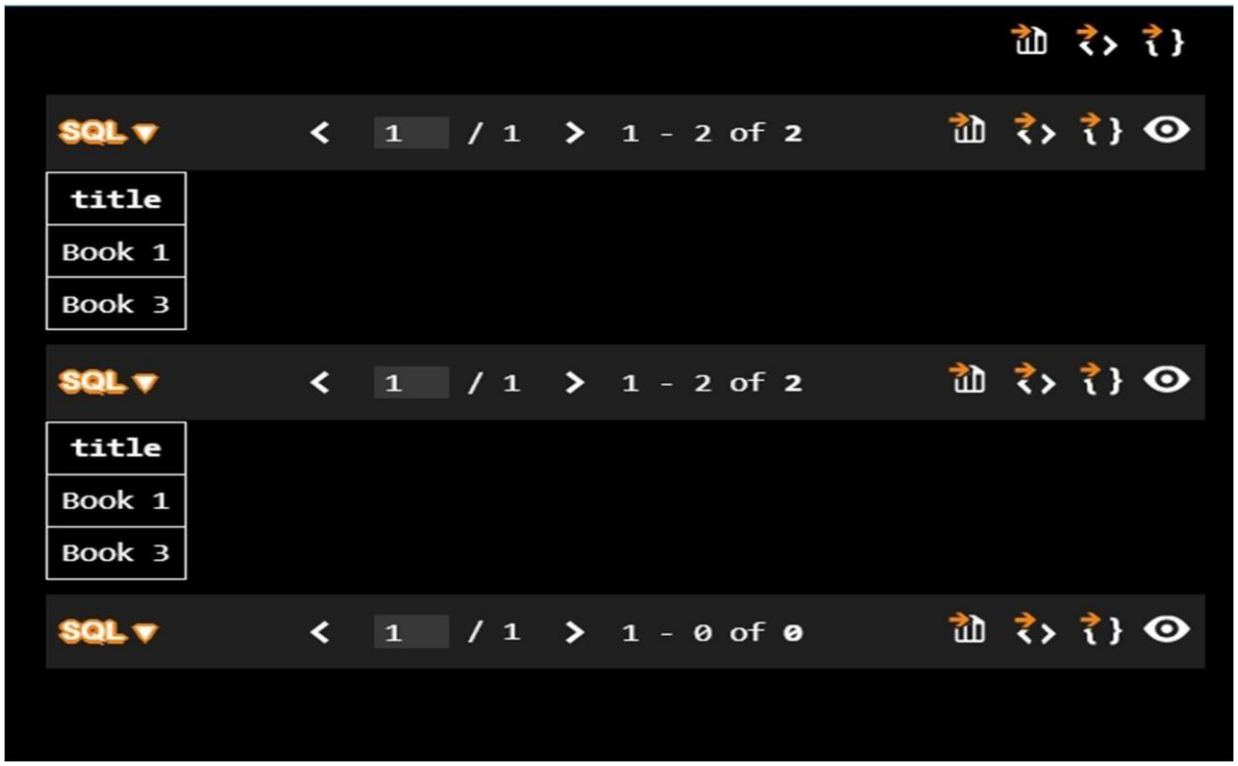
```sql
EXCEPT

SELECT b.title
FROM Books b
JOIN Authors a ON b.author_id = a.id WHERE a.country =
'UK';

-- Indexing for optimization
CREATE INDEX idx_books_author_id ON Books(author_id);
CREATE INDEX idx_authors_country ON Authors(country);

-- Explanation:
-- The original query uses a subquery which may scan data multiple times.  --
The optimized query uses JOIN, allowing faster execution and better use of
indexes.
```

Output:



Explanaion:

The original query uses a subquery, which can be slower for large datasets, while the op2mized query replaces it with a join for faster execu2on. Indexing and reducing nested queries improve performance without changing the result. **Task 5: University Course Registraion Prompt:**

Generate a complete SQL solu2on for a University Course Registra2on System by crea2ng a normalized database schema with four tables: Students, Courses, Faculty, and Registra2ons. Include primary keys, foreign keys, appropriate data types, and constraints such as NOT NULL and UNIQUE, ensuring compa2bility with SQLite. Define rela2onships such that each student can register for mul2ple courses and each course is taught by one faculty member. Maintain at least Third Normal Form (3NF). Then write SQL queries using joins to list all students enrolled in a specific course using course_id, find faculty

members teaching more than 2 courses, and retrieve courses with the highest number of registra2ons. Include sample INSERT statements for tes2ng and ensure the output is clean, well- structured, and fully executable without errors.

Code:

```
-- SQLite
--Generate a complete SQL solution for a University Course Registration System by creating
a normalized database schema with four tables: Students, Courses, Faculty, and
Registrations. Include primary keys, foreign keys, appropriate data types, and
constraints such as NOT NULL and UNIQUE, ensuring compatibility with SQLite. Define
relationships such that each student can register for multiple courses and each course
is taught by one faculty member. Maintain at least Third Normal Form (3NF). Then write
SQL queries using joins to list all students enrolled in a specific course using
course_id, find faculty members teaching more than 2 courses, and retrieve courses with
the highest number of registrations. Include sample INSERT statements for testing and
ensure the output is clean, well-structured, and fully executable without errors.
-- Create the Students table
CREATE TABLE Students ( student_id INTEGER
    PRIMARY KEY AUTOINCREMENT, first_name TEXT
    NOT NULL, last_name TEXT NOT NULL, email
    TEXT NOT NULL UNIQUE
);
-- Create the Faculty table
CREATE TABLE Faculty ( faculty_id INTEGER
    PRIMARY KEY AUTOINCREMENT, first_name TEXT
    NOT NULL, last_name TEXT NOT NULL, email
    TEXT NOT NULL UNIQUE
);
-- Create the Courses table
CREATE TABLE Courses ( course_id INTEGER PRIMARY
    KEY AUTOINCREMENT, course_name TEXT NOT
    NULL, faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);
-- Create the Registrations table
CREATE TABLE Registrations ( registration_id INTEGER PRIMARY
    KEY AUTOINCREMENT, student_id INTEGER NOT
    NULL, course_id INTEGER NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
);
-- Sample INSERT statements for testing
INSERT INTO Faculty (first_name, last_name, email) VALUES ('John', 'Doe',
'john.doe@university.edu');
INSERT INTO Students (first_name, last_name, email) VALUES ('Alice', 'Smith',
'alice.smith@university.edu');
INSERT INTO Courses (course_name, faculty_id) VALUES ('Introduction to Computer
Science', 1);
INSERT INTO Registrations (student_id, course_id) VALUES (1, 1);
```

```sql
-- Query to list all students enrolled in a specific course using course_id
SELECT s.first_name, s.last_name, s.email
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
WHERE r.course_id = 1;
-- Query to find faculty members teaching more than 2 courses
SELECT f.first_name, f.last_name, COUNT(c.course_id) AS course_count
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING course_count > 2;
-- Query to retrieve courses with the highest number of registrations SELECT
c.course_name, COUNT(r.registration_id) AS registration_count
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY registration_count DESC
LIMIT 1;
```

Output:

SQL < 1 / 1 > 1 - 1 of 1

first_name	last_name	email
Alice	Smith	alice.smith@university.edu

SQL < 1 / 1 > 1 - 0 of 0

SQL < 1 / 1 > 1 - 1 of 1

course_name	registration_count
Introduction to Computer Science	1

ExplanaIon:

The Registra2ons table connects Students and Courses, handling the many-to-many rela2onship efficiently. SQL joins and aggrega2on func2ons are used to analyze enrollments, faculty workload, and popular courses.